



**Faculty of Technology and Engineering**

# **DATA STRUCTURES AND ALGORITHMS LABORATORY MANUAL**

Program: B.Tech (CSE / CE / IT / AIML)

Course: CSUC201 – Fundamentals of Data Structures and Algorithms

Semester: III

Academic Year: 2026–27

## Course Outcomes

**CO1:** Understand different types of data structures, analyze upper bound of algorithms, and represent data structures in memory.

**CO2:** Implement linear and non-linear data structures, along with understanding their practical applications.

**CO3:** Perform searching and sorting efficiently, understand upper bound of sorting algorithms, and apply different sorting algorithms to real-world problems.

**CO4:** Implement and analyze hash tables, understand their significance, and use them in real-world applications such as dictionaries.

**CO5:** Develop the skills to select suitable data structures for addressing complex computational challenges.

## Laboratory Policies and Evaluation Scheme

Each laboratory session is of 2 hours duration for single-session practicals and 4 hours for double-session practicals (see individual practical headers). Students are expected to perform the assigned experiment (main problems) and answer the key questions/analysis and viva questions; these are compulsory. The supplementary problems and the Post Laboratory Work (LeetCode practice) are provided for additional practice and are not compulsory.

### Evaluation Scheme

Each practical is evaluated out of 10 marks using a weighted rubric specific to that practical (provided at the end of each practical in this manual). Unlike a flat generic rubric, the weighting reflects the nature of the exercise — correctness, conceptual articulation, comparative reasoning, and code quality — consistently distributed as follows across all practicals:

| Evaluation Criteria (General Pattern)                        | Typical Weight |
|--------------------------------------------------------------|----------------|
| Correctness of program output, including edge cases          | 40%            |
| Conceptual explanation of logic, approach, and optimizations | 25%            |
| Comparative/alternative-strategy reasoning                   | 20%            |
| Code structure, naming, and documentation                    | 15%            |

Exact wording of each criterion is tailored per practical and given in full under each practical's Rubric section.

## Table of Contents

| Practical No. | Aim / Title                                                                      | CO Mapping | Page No. |
|---------------|----------------------------------------------------------------------------------|------------|----------|
| 1             | Array Operations: Traversal, Rotation, and Frequency Analysis                    | CO1, CO2   | 05       |
| 2             | Linear and Binary Search: Iterative and Recursive Approaches                     | CO1, CO3   | 08       |
| 3             | Sorting Algorithms: Comparison-Based, Counting-Based, and Digit-Based Strategies | CO1, CO3   | 11       |
| 4             | Singly Linked List: Dynamic Insertion, Deletion, and Traversal                   | CO1, CO2   | 14       |
| 5             | Doubly Linked List and Circular Linked List Operations                           | CO2, CO5   | 17       |
| 6             | Stack Implementation using Array and Linked List, with Expression Processing     | CO2, CO5   | 20       |
| 7             | Queue Implementation using Array and Linked List, with Structure Simulation      | CO2, CO5   | 23       |
| 8             | Binary Tree Traversals and Binary Search Tree Insertion                          | CO1, CO2   | 26       |
| 9             | Graph Traversal: Depth-First Search and Breadth-First Search                     | CO1, CO2   | 29       |
| 10            | Hash Tables: Linear Probing, Chaining, and Double Hashing                        | CO1, CO4   | 32       |

## Practical 1: Array Operations: Traversal, Rotation, and Frequency Analysis

|                                                         |                                                                                                                                                                                    |
|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CO Mapping</b>                                       | CO1 – Analyse array representation in memory and reason about algorithmic upper bounds;<br>CO2 – Implement linear data structure operations and apply them to practical scenarios. |
| <b>Bloom's Taxonomy Level</b>                           | L3 – Apply / L4 – Analyze                                                                                                                                                          |
| <b>Lab Duration</b>                                     | 2 Hours                                                                                                                                                                            |
| <b>Total Hours of Problem Definition Implementation</b> | 1.5 Hours                                                                                                                                                                          |
| <b>Total Hours of Testing / Evaluation</b>              | 0.5 Hours                                                                                                                                                                          |

### Practical Aim

To implement fundamental array operations.

#### Problem 1

A bakery prepares  $n$  items every morning and places them in a display row. At the end of each hour, the leftmost item is moved to the rightmost position to make room for fresh stock at the front. Given the initial row and the number of hours  $h$ , print the final display order.

Describe the approach you used. How does it behave when  $h$  is very large, say 10 million? Is there a way to get the correct result without performing the operation  $h$  times?

#### Problem 2

A library issues books to students and records each book's ID every time it is borrowed. At the end of the month, the librarian wants to find all books that were borrowed more than once, as those need priority restocking. Given the borrowing log, print all such book IDs.

Describe the approach you used. How many times does your solution scan the data? If the library had 100,000 borrow records, would your approach still be practical? Can you think of a way that requires fewer passes?

#### Problem 3

A school newspaper editor wants to highlight the most impressive word from a submitted article on the front page. The word is chosen simply by length — the longest one wins. Given a sentence, print the winning word and how many letters it has.

Describe the approach you used. What does your solution do when two words have the same length? Is that behavior intentional, and can you think of a way to handle it explicitly?

### Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] The display order repeats itself after a certain number of hours regardless of how large  $h$  gets. Without trying any specific example, explain why this repetition is inevitable and what determines when it first repeats.

2. [Problem 2] If the same book ID appears three times in the log, should it appear once or three times in your output? Does your solution handle this correctly, and how would you verify it?
3. [Problem 3] The editor later decides that punctuation attached to a word (e.g. “hello,” or “great!”) should not count toward its length. How would this change your solution, and at which step would you handle it?

## Supplementary Problems

1. A school has classrooms numbered 1 to  $n$ . After renovation, workers recorded which rooms are now ready. But the site manager suspects some rooms were skipped and never reported. Given the list of reported room numbers, find all the room numbers that are missing from the report.

Your solution probably checks every number from 1 to  $n$  against the list one by one. Can you think of a mathematical shortcut — something involving the expected total — that could help you find missing numbers faster?

2. A polling booth records the ID of every voter as they cast their vote. After polling ends, officials want to identify voters who voted exactly once — since anyone appearing more than once in the log is a duplicate entry that must be removed. Given the voter log, print all IDs that appear exactly once.

Your solution likely counts occurrences by scanning the full log for each voter ID. If the log has thousands of entries, this gets slow. Can you think of a way to count all IDs in a single pass through the log?

## Key Skills to be Addressed

Array traversal and index manipulation, frequency counting and duplicate detection, string tokenization, edge case handling, and conceptual reasoning about time complexity.

## Applications

Circular buffers and job scheduling (rotation), inventory and log deduplication, text analysis, and audit or data integrity checks.

## Learning Outcome

Students will be able to apply array operations to solve real-world problems, identify inefficiencies in naive approaches, and articulate alternative strategies even when full implementation is beyond current scope.

## Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design suitable test cases covering typical inputs, boundary/edge cases, and at least one stress/worst-case scenario. Document the selected inputs and expected outputs before implementation.

## Tools / Technology to be Used

C / C++, VS Code.

## Total Hours of Engagement

2 Hours (1.5 hrs implementation, 30 min testing/modification/faculty evaluation)

## Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems. The Supplementary Problems and LeetCode practice set below are for additional practice and are not required for submission.

## Post Laboratory Work (LeetCode Practice)

| Sr. # | LC # | Problem                      | Concept                                    |
|-------|------|------------------------------|--------------------------------------------|
| 1     | 485  | Max Consecutive Ones         | Array traversal and counting               |
| 2     | 238  | Product of Array Except Self | Array manipulation without named technique |
| 3     | 560  | Subarray Sum Equals K        | Frequency reasoning on arrays              |
| 4     | 287  | Find the Duplicate Number    | Duplicate detection with constraints       |
| 5     | 41   | First Missing Positive       | Missing element under constraints          |

## Rubrics (Practical Evaluation / Viva)

| Criteria                                                                                             | Weight (%) |
|------------------------------------------------------------------------------------------------------|------------|
| Program produces correct output for all given and edge-case inputs.                                  | 40%        |
| Student can explain their logic, choice of approach, and any optimizations applied.                  | 25%        |
| Student can describe at least one alternative strategy in plain English and identify its trade-offs. | 20%        |
| Code is well-structured, uses meaningful variable names, and includes basic comments.                | 15%        |

## Practical 2: Linear and Binary Search: Iterative and Recursive Approaches

|                                                         |                                                                                                                                                                     |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CO Mapping</b>                                       | CO1 – Analyse algorithmic upper bounds of search strategies;<br>CO3 – Perform searching efficiently and apply appropriate search algorithms to real-world problems. |
| <b>Bloom's Taxonomy Level</b>                           | L3 – Apply / L4 – Analyze                                                                                                                                           |
| <b>Lab Duration</b>                                     | 2 Hours                                                                                                                                                             |
| <b>Total Hours of Problem Definition Implementation</b> | 1.5 Hours                                                                                                                                                           |
| <b>Total Hours of Testing / Evaluation</b>              | 0.5 Hours                                                                                                                                                           |

### Practical Aim

To implement linear and binary search using both iterative and recursive approaches.

#### Problem 1

A security guard at a parking lot checks vehicles one by one from the entrance to find a car with a specific license plate. Sometimes he starts from the entrance, sometimes he calls a helper who starts from where the guard left off. Given a list of license plates and a target plate, implement both approaches — one that checks plates one by one from the start, and one where the function calls itself to continue checking — and report the position of the target plate if found.

Describe the approach you used. What happens in your solution if the target plate appears more than once in the list? Does it find the first occurrence, the last, or just any one?

#### Problem 2

A librarian maintains a sorted catalog of book codes and needs to locate a specific code quickly. Instead of going through every book, she opens the catalog to the middle, checks whether the target is to the left or right, and repeats. Given a sorted list of book codes and a target code, implement both approaches — one using a loop and one where the function calls itself — and report the position of the target code.

Describe the approach you used. What is the one condition your input must satisfy for this approach to work correctly? What would go wrong if that condition was not met?

### Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] If the target plate appears near the end of a very long list, your solution still checks every plate before it. Can you think of any property the input would need to have for you to skip checking some plates entirely? What does Problem 2.2 tell you about this?
2. [Problem 2] What is the consequence of applying your solution to a list that is not sorted? Would it always give a wrong answer, or only sometimes? Explain your reasoning without running any example.

## Supplementary Problems

1. A school assigns roll numbers to students starting from 1. Due to some admissions being cancelled, some roll numbers in the middle are missing. Given a sorted list of assigned roll numbers and a number  $k$ , find the  $k$ th roll number that was never assigned. Describe the approach you used. Does your solution work correctly when  $k$  is larger than the total number of missing numbers in the visible list? How would you handle that?
2. A warehouse stores items in a grid where quantities increase from left to right along each row and from top to bottom along each column. Given this grid and a target quantity, determine whether that quantity exists anywhere in the grid. Describe the approach you used. If you checked every cell in the grid, how many cells would that be for a  $100 \times 100$  grid? Can you think of a smarter starting point that lets you eliminate an entire row or column at each step?

## Key Skills to be Addressed

Linear and binary search using iteration and recursion, search space elimination, handling sorted vs. unsorted data, and recognising when a precondition must hold for an algorithm to work correctly.

## Applications

Record lookup in databases, catalog and inventory search, range queries in sorted data, and grid-based search in logistics and mapping systems.

## Learning Outcome

Students will be able to implement search algorithms iteratively and recursively, understand why the same problem can be solved in more than one way, and reason about which approach is better suited given the structure of the input data.

## Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design suitable test cases covering typical inputs, boundary/edge cases, and at least one stress/worst-case scenario. Document the selected inputs and expected outputs before implementation.

## Tools / Technology to be Used

C / C++, VS Code.

## Total Hours of Engagement

2 Hours (1.5 hrs implementation, 30 min testing/modification/faculty evaluation)

## Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems. The Supplementary Problems and LeetCode practice set below are for additional practice and are not required for submission.

## Post Laboratory Work (LeetCode Practice)

| Sr. # | LC # | Problem                | Concept                       |
|-------|------|------------------------|-------------------------------|
| 1     | 35   | Search Insert Position | Binary search on sorted array |

| Sr. # | LC # | Problem                        | Concept                                    |
|-------|------|--------------------------------|--------------------------------------------|
| 2     | 374  | Guess Number Higher or Lower   | Recursive binary search logic              |
| 3     | 33   | Search in Rotated Sorted Array | Binary search under modified conditions    |
| 4     | 162  | Find Peak Element              | Search without full sorted order           |
| 5     | 74   | Search a 2D Matrix             | 2D binary search with row-column structure |

### Rubrics (Practical Evaluation / Viva)

| Criteria                                                                                             | Weight (%) |
|------------------------------------------------------------------------------------------------------|------------|
| Program produces correct output for all given and edge-case inputs.                                  | 40%        |
| Student can explain their logic, choice of approach, and any optimizations applied.                  | 25%        |
| Student can describe at least one alternative strategy in plain English and identify its trade-offs. | 20%        |
| Code is well-structured, uses meaningful variable names, and includes basic comments.                | 15%        |

## Practical 3: Sorting Algorithms: Comparison-Based, Counting-Based, and Digit-Based Strategies

|                                                         |                                                                                                                                                            |
|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CO Mapping</b>                                       | CO1 – Analyse the upper bound of sorting algorithms;<br>CO3 – Apply different sorting algorithms to real-world problems and reason about their efficiency. |
| <b>Bloom's Taxonomy Level</b>                           | L4 – Analyze / L5 – Evaluate                                                                                                                               |
| <b>Lab Duration</b>                                     | 2 Hours                                                                                                                                                    |
| <b>Total Hours of Problem Definition Implementation</b> | 1.5 Hours                                                                                                                                                  |
| <b>Total Hours of Testing / Evaluation</b>              | 0.5 Hours                                                                                                                                                  |

### Practical Aim

To implement and compare multiple sorting algorithms and develop the ability to reason about which sorting strategy suits a given situation.

#### Problem 1

A teacher has a stack of student answer sheets with marks written on them and needs to arrange them in order before entering grades. She tries three different methods: in the first, she repeatedly compares adjacent sheets and swaps them if they are out of order; in the second, she finds the lowest-marked sheet each time and places it at the front; in the third, she picks each sheet one by one and inserts it into its correct position among the already-arranged sheets. Implement all three methods and for each one, print the sorted order of marks.

Describe how each of the three methods works in your own words. If the sheets were already in order before the teacher started, which method would finish the fastest and why?

#### Problem 2

A paint shop has buckets labeled with one of three colour codes — 0, 1, or 2 — but they are stored in a random order. The shop owner wants all 0s together, then all 1s, then all 2s, without using any extra storage. Given the list of colour codes, rearrange them in place and print the result.

Describe the approach you used. Does your solution make more than one pass through the list? Can you think of a way to do it in exactly one pass?

### Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

- [Problem 1] One of the three methods can detect early that the list is already sorted and stop without doing any more work. Which method is it, how would you add that detection, and why can the other two not do the same?
- [Problem 2] If your solution makes two passes through the list, can you identify which decision in your logic forces the second pass? Is there a way to restructure that decision to eliminate it?

## Supplementary Problems

1. A school is printing certificates for students and the printer needs roll numbers sorted before it can print in order. All roll numbers are between 1 and 100. Rather than comparing roll numbers against each other, the organiser wants to count how many times each roll number appears and use those counts to reconstruct the sorted list. Given a list of roll numbers, sort them using this counting-based approach and print the result.

Describe the approach you used. What would break in your solution if one of the roll numbers was 5000 instead of being between 1 and 100? Can you think of why that matters?

2. A courier company needs to sort thousands of parcel tracking codes, all of which are 6-digit numbers. Instead of comparing the full number each time, a sorter processes the digits one position at a time — first sorting by the last digit, then the second last, and so on — until all codes are in order. Given a list of tracking codes, sort them using this digit-by-digit approach and print the result.

Describe the approach you used. Your solution processes digits from right to left — what would go wrong if you processed them from left to right instead? Try a small example to verify your answer.

## Key Skills to be Addressed

Comparison-based sorting using multiple strategies, in-place rearrangement, counting-based sorting, digit-wise sorting, and reasoning about which method suits a given input type.

## Applications

Grade and record ordering, inventory and warehouse management, certificate and document processing, and logistics tracking systems.

## Learning Outcome

Students will be able to implement multiple sorting algorithms, trace their execution step by step, compare their behaviour on different inputs, and articulate why certain sorting approaches work only under specific input conditions.

## Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design suitable test cases covering typical inputs, boundary/edge cases, and at least one stress/worst-case scenario. Document the selected inputs and expected outputs before implementation.

## Tools / Technology to be Used

C / C++, VS Code.

## Total Hours of Engagement

2 Hours (1.5 hrs implementation, 30 min testing/modification/faculty evaluation)

## Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems. The Supplementary Problems and LeetCode practice set below are for additional practice and are not required for submission.

## Post Laboratory Work (LeetCode Practice)

| Sr. # | LC # | Problem                             | Concept                           |
|-------|------|-------------------------------------|-----------------------------------|
| 1     | 164  | Maximum Gap                         | Sorting with constraints          |
| 2     | 274  | H-Index                             | Counting-based sorting logic      |
| 3     | 451  | Sort Characters By Frequency        | Frequency-driven ordering         |
| 4     | 179  | Largest Number                      | Custom comparator sorting         |
| 5     | 315  | Count of Smaller Numbers After Self | Sorting with positional reasoning |

## Rubrics (Practical Evaluation / Viva)

| Criteria                                                                                                                                     | Weight (%) |
|----------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Program produces correct output for all three sorting methods and handles edge cases such as already-sorted or all-same-value inputs.        | 40%        |
| Student can trace through at least one sorting method step by step and explain why each swap, selection, or insertion happens.               | 25%        |
| Student can describe in plain English how a different sorting method would approach the same input and identify one advantage it would have. | 20%        |
| Code is well-structured, each sorting method is clearly separated, uses meaningful variable names, and includes basic comments.              | 15%        |

## Practical 4: Singly Linked List: Dynamic Insertion, Deletion, and Traversal

|                                                         |                                                                                                                                                                           |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CO Mapping</b>                                       | CO1 – Represent dynamic data structures in memory using pointer-based linking;<br>CO2 – Implement linear non-array data structures and apply them to practical scenarios. |
| <b>Bloom's Taxonomy Level</b>                           | L3 – Apply / L4 – Analyze                                                                                                                                                 |
| <b>Lab Duration</b>                                     | 4 Hours                                                                                                                                                                   |
| <b>Total Hours of Problem Definition Implementation</b> | 3.5 Hours                                                                                                                                                                 |
| <b>Total Hours of Testing / Evaluation</b>              | 0.5 Hours                                                                                                                                                                 |

### Practical Aim

To implement core singly linked list operations and develop the ability to manipulate dynamic sequences through real-world problem scenarios.

#### Problem 1

A hospital manages a queue of patient tokens. New critical patients must be added to the front, routine patients are added to the end, and occasionally a patient with a priority number must be inserted at a specific position in the queue. Given a sequence of such operations, implement all three insertion types and print the final queue after each operation.

Describe how your insertion at a specific position works. What does your solution do if the position given is greater than the current length of the queue? Is that handled explicitly or does it cause an error?

#### Problem 2

The hospital also needs to remove a patient token from the queue when a patient leaves, print all remaining tokens from last to first for an end-of-day audit, and display the full queue from front to back at any point. Implement deletion by value, reverse printing, and forward traversal on the same queue from Problem 4.1a.

Describe how your reverse printing works. Does it use any extra storage, or does it work purely by following the links? Can you think of another way to reverse-print without modifying the original queue?

### Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] Inserting at a position beyond the current length of the list is an invalid operation. Should your solution silently insert at the end, report an error, or do something else entirely? Justify your choice and explain what assumption about the caller your choice makes.
2. [Problem 2] If the node to be deleted is the only node in the list, how many pointers need to change and what must they be set to? How is this case structurally different from deleting a node in the middle?

## Supplementary Problems

1. A train has  $n$  coaches numbered in order. The train manager wants to detach the coach that is exactly  $k$  positions from the rear of the train, without counting from the front. Given the list of coach numbers and the value  $k$ , remove the correct coach and print the remaining sequence.

Describe the approach you used to find the correct coach without knowing the total number of coaches in advance. What happens if  $k$  equals the total number of coaches?

2. A dance instructor pairs up students standing in a line and swaps each pair — the first with the second, the third with the fourth, and so on. Given the list of student IDs, swap every two adjacent students and print the new order. If the total number of students is odd, the last student stays in place.

Describe how your solution handles the swapping. Does it swap the actual node positions or just the values inside the nodes? What is the difference, and does it matter here?

3. A relay race has runners lined up in a chain. The coach wants to identify the runner standing exactly in the middle of the chain to give them a special baton. Given the list of runner IDs, find and print the middle runner's ID. If there are two middle runners, print the second one.

Describe the approach you used. Does your solution make more than one pass through the list? Can you think of a way to find the middle in a single pass without knowing the total count first?

4. A film strip has frames stored in a linked sequence. A technician needs to reverse the order of all frames so the film plays backwards. Given the list of frame numbers, reverse the sequence and print the result.

Describe how your solution reverses the links. How many passes through the list does it make, and how many extra nodes does it create? Can you think of a way to do it using recursion instead?

5. A message is stored as a sequence of characters in a linked chain. A decoder needs to verify whether the message reads the same forwards and backwards before accepting it. Given the sequence of characters, determine whether it forms a palindrome and print Yes or No.

Describe the approach you used. Does your solution modify the original sequence at any point? If yes, does it restore it before finishing, and why might that matter?

## Key Skills to be Addressed

Dynamic node insertion and deletion, pointer/link manipulation, sequence traversal and reversal, two-pointer reasoning, and handling edge cases in dynamic data structures.

## Applications

Task and patient queue management, undo/redo sequences in editors, playlist and media management, and message validation systems.

## Learning Outcome

Students will be able to implement and manipulate singly linked lists through pointer-based operations, trace link changes step by step, and reason about the trade-offs between approaches that modify structure versus those that use extra storage.

## Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design suitable test cases covering typical inputs, boundary/edge cases, and at least one stress/worst-case scenario. Document the selected inputs and expected outputs before implementation.

## Tools / Technology to be Used

C / C++, VS Code.

## Total Hours of Engagement

4 Hours (3.5 hrs implementation, 30 min testing/modification/faculty evaluation)

## Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems. The Supplementary Problems and LeetCode practice set below are for additional practice and are not required for submission.

## Post Laboratory Work (LeetCode Practice)

| Sr. # | LC # | Problem              | Concept                              |
|-------|------|----------------------|--------------------------------------|
| 1     | 328  | Odd Even Linked List | Node regrouping by position          |
| 2     | 148  | Sort List            | Sorting applied to linked list       |
| 3     | 143  | Reorder List         | Multi-step linked list restructuring |
| 4     | 2    | Add Two Numbers      | Positional arithmetic on linked list |
| 5     | 61   | Rotate List          | Rotation applied to linked list      |

## Rubrics (Practical Evaluation / Viva)

| Criteria                                                                                                                                                              | Weight (%) |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Program correctly performs all linked list operations including edge cases such as empty list, single node, and invalid position or value.                            | 40%        |
| Student can trace through a pointer change step by step — identifying which node's link changes, what it pointed to before, and what it points to after.              | 25%        |
| Student can describe an alternative way to perform at least one operation (e.g. reverse printing with vs. without extra storage) and identify what is gained or lost. | 20%        |
| Code is well-structured with clearly separated functions for each operation, meaningful variable names, and basic comments on pointer manipulation steps.             | 15%        |

## Practical 5: Doubly Linked List and Circular Linked List Operations

|                                                         |                                                                                                                                                                                                          |
|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CO Mapping</b>                                       | CO2 – Implement non-linear linking strategies and understand their practical applications;<br>CO5 – Select a suitable linked structure (singly, doubly, or circular) for a given computational scenario. |
| <b>Bloom's Taxonomy Level</b>                           | L4 – Analyze                                                                                                                                                                                             |
| <b>Lab Duration</b>                                     | 4 Hours                                                                                                                                                                                                  |
| <b>Total Hours of Problem Definition Implementation</b> | 3.5 Hours                                                                                                                                                                                                |
| <b>Total Hours of Testing / Evaluation</b>              | 0.5 Hours                                                                                                                                                                                                |

### Practical Aim

To implement core doubly linked list and circular linked list operations and understand how bidirectional and circular linking change the way nodes are inserted, deleted, and traversed.

#### Problem 1

A music player maintains a playlist where songs can be added to the beginning, added to the end, or inserted right after a specific song that is currently playing. When a song is removed, only the first song in the playlist is dropped. At any point the player can count how many songs are in the playlist and display them from first to last. Given a sequence of such operations, implement all of them on a doubly linked playlist and print the result after each operation.

Describe how your insertion after a given song works. What extra step does a doubly linked list require here that a singly linked list did not? What happens if the given song is not found in the playlist?

#### Problem 2

A group of students are sitting in a circle playing a passing game. A token starts at the first student and is passed around the circle one student at a time. Students can join the circle at any position and leave at any time, and the circle must remain unbroken after every join or leave. Given a sequence of join, leave, and display operations, implement them on both a singly circular and a doubly circular linked list and print the current circle after each operation.

Describe how your solution ensures the circle stays unbroken after a student leaves. What is the one case where breaking the circle is most likely to happen as a bug, and how did you handle it?

### Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] A doubly linked list maintains two links per node instead of one. When deleting the front node, explain why having a backward link on the second node creates a responsibility that did not exist in a singly linked list — and what goes wrong if you forget to handle it.

2. [Problem 2] If your traversal of the circular list does not stop at the right moment, it will loop forever. Without referencing any specific code, explain what condition must be true at the moment you stop, and why that condition would fail if you set the last node's link to null.

## Supplementary Problems

1. A hospital maintains two separate ward admission lists, each already arranged in a particular order. At the end of the day, the administrator needs to combine both lists into a single unified list while preserving the same order. Given the two lists, merge them into one correctly ordered list and print the result.

Describe the approach you used to merge the two lists. Did you notice any property of the two input lists that your solution depends on? What would happen to your output if that property did not hold?

2. A supermarket loyalty system stores a linked list of customer IDs collected during the day. Due to a scanner glitch, some customer IDs were recorded multiple times. The system needs to remove every occurrence of a given ID from the list, not just the first one. Given the list and a target ID, remove all its occurrences and print the remaining list.

Describe how your solution handles the case where the target ID appears at the front, in the middle, and at the end of the list. Does your solution handle all three cases correctly with the same logic, or did you need to treat them separately?

3. A circular bus route has stops connected in a loop. A dispatcher wants to know how many stops are on the route and print them all starting from a given stop, going around once until returning to the start.

Describe how your solution knows when to stop traversing. What condition tells your program that it has gone around the full circle exactly once? What would happen if that condition was slightly wrong?

## Key Skills to be Addressed

Bidirectional link manipulation, circular link maintenance, forward and backward pointer updates, sequential merging, multi-occurrence deletion, and termination condition reasoning in circular structures.

## Applications

Music and media playlist management, round-robin scheduling and token passing systems, hospital queue administration, browser history navigation, and data deduplication in record systems.

## Learning Outcome

Students will be able to implement doubly and circular linked list operations, articulate how maintaining two links or a circular connection changes insertion, deletion, and traversal logic, and handle edge cases such as broken circles, missing nodes, and repeated values in dynamic sequences.

## Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design suitable test cases covering typical inputs, boundary/edge cases, and at least one stress/worst-case scenario. Document the selected inputs and expected outputs before implementation.

## Tools / Technology to be Used

C / C++, VS Code.

## Total Hours of Engagement

4 Hours (3.5 hrs implementation, 30 min testing/modification/faculty evaluation)

## Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems. The Supplementary Problems and LeetCode practice set below are for additional practice and are not required for submission.

## Post Laboratory Work (LeetCode Practice)

| Sr. # | LC # | Problem                                  | Concept                                      |
|-------|------|------------------------------------------|----------------------------------------------|
| 1     | 1472 | Design Browser History                   | Doubly linked bidirectional navigation       |
| 2     | 430  | Flatten a Multilevel Doubly Linked List  | Doubly linked pointer manipulation           |
| 3     | 141  | Linked List Cycle                        | Cycle detection in circular structures       |
| 4     | 142  | Linked List Cycle II                     | Finding cycle entry point                    |
| 5     | 426  | Convert BST to Sorted Doubly Linked List | DLL pointer manipulation with tree structure |

## Rubrics (Practical Evaluation / Viva)

| Criteria                                                                                                                                                                                  | Weight (%) |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Program correctly performs all doubly and circular linked list operations including edge cases such as empty list, single node, last node removal, and target not found.                  | 40%        |
| Student can identify which specific links change during an insertion or deletion in both doubly and circular structures and trace through the pointer updates step by step.               | 25%        |
| Student can explain how the same operation would differ between a singly linked, doubly linked, and circular linked list and articulate what each structural difference enables or costs. | 20%        |
| Code is well-structured with clearly separated functions for each list type and operation, meaningful variable names, and comments on pointer updates.                                    | 15%        |

## Practical 6: Stack Implementation using Array and Linked List, with Expression Processing

|                                                         |                                                                                                                                                                                                                              |
|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CO Mapping</b>                                       | CO2 – Implement a linear data structure (stack) using two underlying representations;<br>CO5 – Select a suitable underlying representation and apply stack-based reasoning to expression conversion and validation problems. |
| <b>Bloom's Taxonomy Level</b>                           | L3 – Apply / L4 – Analyze                                                                                                                                                                                                    |
| <b>Lab Duration</b>                                     | 4 Hours                                                                                                                                                                                                                      |
| <b>Total Hours of Problem Definition Implementation</b> | 3.5 Hours                                                                                                                                                                                                                    |
| <b>Total Hours of Testing / Evaluation</b>              | 0.5 Hours                                                                                                                                                                                                                    |

### Practical Aim

To implement stack using array and linked list, and apply stack-based logic to expression conversion, evaluation, and bracket validation problems.

#### Problem 1

A cafeteria stacks clean trays on a fixed-size counter. New trays are always placed on top, and customers always take from the top. The counter can hold at most  $n$  trays at a time — if it is full, no more trays can be added, and if it is empty, no tray can be taken. Given a sequence of place and take operations, implement this fixed-capacity tray stack and print the current top tray after each operation. Report an error if a place or take operation cannot be performed.

Describe how your solution tracks whether the stack is full or empty. What are the exact conditions you check before a push and before a pop, and what does your program do when those conditions fail?

#### Problem 2

A web browser keeps track of pages visited so that the back button always returns to the most recently visited page. Unlike a fixed counter, the browser has no hard limit on how many pages it can remember — it grows as the user visits more pages and shrinks as they press back. Given a sequence of visit and back operations, implement this unlimited page history and print the current page after each operation.

Describe how your solution differs from Problem 1 in how it handles memory. What happens in your solution when the user presses back on the very first page with no history left?

#### Problem 3

A calculator application receives arithmetic expressions typed by users in the usual way — numbers and operators written in between, with brackets to control priority (e.g.  $3 + 4 * 2$  or  $(3 + 4) * 2$ ). Internally the calculator needs to convert each expression into a form where operators appear after their operands, so it can evaluate them without needing to re-read brackets. Given an infix expression, convert it to the equivalent postfix form and print the result.

Describe the role the stack plays in your conversion. What does your solution do when it encounters a closing bracket, and what happens to the operators that were waiting on the stack at that point?

### **Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation**

1. [Problem 1] A circular array is one way to reuse freed slots without shifting all elements. Without implementing it, explain what change would need to be made to your front and rear pointer logic to make the array behave circularly, and what new condition would indicate a full array.
2. [Problem 2] Your linked list stack has no fixed capacity limit. But in practice, can it still run out of space? What determines the actual limit, and why is that limit invisible in your code?
3. [Problem 3] operator precedence determines the order in which operators are placed into the output. Without tracing any expression, explain what would go wrong in your output if your solution treated all operators as having equal precedence.

### **Supplementary Problems**

1. The same calculator from Problem 3 now receives the postfix expression it produced and needs to compute the final numeric result. Operands are pushed onto a stack, and each operator causes the top two operands to be popped, combined, and pushed back. Given a postfix expression with single-digit operands, evaluate it and print the final result.

Describe how your solution handles an operator in the postfix expression. What exactly gets popped, what operation is applied, and what gets pushed back? What would happen if the expression was malformed and an operator appeared before enough operands were on the stack?

2. A code editor checks every file for correctly matched brackets before compiling. Every opening bracket must have a corresponding closing bracket of the same type, and they must be properly nested — a bracket opened later must be closed before one opened earlier. Given a string of brackets, determine whether the sequence is valid and print Yes or No.

Describe the approach you used. What does your solution do when it encounters a closing bracket, and how does it know whether that closing bracket matches the correct opening bracket? What does your solution output for an empty string?

### **Key Skills to be Addressed**

Stack implementation using arrays and linked lists, operator precedence and associativity handling, expression conversion and evaluation, and bracket matching using stack-based logic.

### **Applications**

Browser history and undo/redo systems, compiler expression parsing, calculator applications, code editors and syntax validators, and any system requiring last-in first-out processing.

### **Learning Outcome**

Students will be able to implement a stack using two different underlying structures, understand the trade-offs between fixed and dynamic capacity, and apply stack-based reasoning to solve expression conversion, evaluation, and validation problems.

## Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design suitable test cases covering typical inputs, boundary/edge cases, and at least one stress/worst-case scenario. Document the selected inputs and expected outputs before implementation.

## Tools / Technology to be Used

C / C++, VS Code.

## Total Hours of Engagement

4 Hours (3.5 hrs implementation, 30 min testing/modification/faculty evaluation)

## Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems. The Supplementary Problems and LeetCode practice set below are for additional practice and are not required for submission.

## Post Laboratory Work (LeetCode Practice)

| Sr. # | LC # | Problem                        | Concept                                 |
|-------|------|--------------------------------|-----------------------------------------|
| 1     | 155  | Min Stack                      | Stack with auxiliary tracking           |
| 2     | 71   | Simplify Path                  | Stack applied to path string processing |
| 3     | 394  | Decode String                  | Nested stack-based processing           |
| 4     | 84   | Largest Rectangle in Histogram | Stack-based span reasoning              |
| 5     | 496  | Next Greater Element I         | Stack applied to sequence comparison    |

## Rubrics (Practical Evaluation / Viva)

| Criteria                                                                                                                                                         | Weight (%) |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Program correctly handles all stack operations including overflow, underflow, expression conversion, evaluation, and bracket validation across all edge cases.   | 40%        |
| Student can trace through the stack state step by step for a given expression or operation sequence and explain what changes at each step and why.               | 25%        |
| Student can explain the difference between array-based and linked-list-based stacks and describe a situation where one would be preferred over the other.        | 20%        |
| Code is well-structured with clearly separated functions for each operation and problem, meaningful variable names, and comments explaining stack state changes. | 15%        |

## Practical 7: Queue Implementation using Array and Linked List, with Structure Simulation

|                                                         |                                                                                                                                                                                                                          |
|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CO Mapping</b>                                       | CO2 – Implement a linear data structure (queue) using two underlying representations;<br>CO5 – Select a suitable underlying representation and apply queue-based reasoning to number generation and simulation problems. |
| <b>Bloom's Taxonomy Level</b>                           | L3 – Apply / L4 – Analyze                                                                                                                                                                                                |
| <b>Lab Duration</b>                                     | 2 Hours                                                                                                                                                                                                                  |
| <b>Total Hours of Problem Definition Implementation</b> | 1.5 Hours                                                                                                                                                                                                                |
| <b>Total Hours of Testing / Evaluation</b>              | 0.5 Hours                                                                                                                                                                                                                |

### Practical Aim

To implement queue using array and linked list, and apply queue-based logic to number generation, and structure simulation problems.

#### Problem 1

A government office has a token counter that issues tokens to visitors. The counter can hold at most  $n$  tokens at a time. New visitors join from one end and are served from the other end in the order they arrived. If the counter is full, no new token can be issued, and if it is empty, no one can be served. Given a sequence of join and serve operations, implement this fixed-capacity token system and print the current front token after each operation. Report an error if an operation cannot be performed.

Describe how your solution tracks the front and rear of the queue. What happens to your front and rear pointers over time as join and serve operations keep happening, and does your solution handle the case where space appears to run out even though slots have been freed?

#### Problem 2

A hospital emergency ward receives patients continuously and attends to them strictly in the order they arrived, with no upper limit on how many can be waiting at once. New patients are added at the back and the doctor always attends to the patient at the front. Given a sequence of arrive and attend operations, implement this unlimited patient queue and print the current front patient after each operation.

Describe how your solution differs from Problem 1 in how it manages capacity. What happens in your solution when the attend operation is called on an empty ward with no patients waiting?

### Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] A simple array queue eventually reports being full even when slots have been freed at the front. Explain why this happens structurally — not by tracing an example, but by describing what the front and rear pointers represent and what their movement pattern causes over time.

2. [Problem 2] Your linked list queue has no capacity limit. If arrive operations vastly outnumber attend operations over a long period, what real consequence would this have on the system, and where in your code is that consequence invisible?

## Supplementary Problems

1. A digital display system needs to show binary representations of numbers from 1 to n in order, without doing any direct binary conversion. Starting from "1", the system generates the next numbers by appending a 0 and then a 1 to each number it has already generated, processing them strictly in the order they were created. Given n, generate and print the binary representations of 1 through n using this generation approach.

Describe how your solution decides what to generate next. What data structure did you use to keep track of the order, and what property of that structure makes this generation approach work correctly?

2. A kitchen has two serving counters that work strictly on a first-come first-served basis, but the chef needs to plate dishes in reverse order — the most recently prepared dish must always go out first. Using only these two counters and their basic operations, implement a system that always serves the most recently added dish first. Given a sequence of add and serve operations, print the dish served after each serve operation.

Describe the approach you used. Which counter holds the dishes most of the time, and what role does the second counter play? What is the cost of your approach in terms of how many times a dish moves between counters?

3. A ticketing booth has two stacks of physical tickets. The booth must always issue tickets in the order they were received — the oldest ticket must go first — but the only available storage units work in a last-in first-out manner. Using only these two stacks and their basic operations, implement a system that always issues the oldest ticket first. Given a sequence of add and issue operations, print the ticket issued after each issue operation.

Describe the approach you used. When does your solution move tickets from one stack to the other, and what triggers that move? Can you think of a way to reduce the number of unnecessary moves between the two stacks?

## Key Skills to be Addressed

Queue implementation using arrays and linked lists, circular array indexing, order-preserving generation, and structure simulation using primitive operations of another structure.

## Applications

Token and ticketing systems, hospital and service queue management, binary number generation in digital systems, and task scheduling in operating systems.

## Learning Outcome

Students will be able to implement a queue using two different underlying structures, understand the trade-offs between fixed and dynamic capacity, and apply queue-based reasoning to solve number generation and structure simulation problems where the order of processing is critical.

## Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design suitable test cases covering typical inputs, boundary/edge cases, and at least one stress/worst-case scenario. Document the selected inputs and expected outputs before implementation.

## Tools / Technology to be Used

C / C++, VS Code.

## Total Hours of Engagement

2 Hours (1.5 hrs implementation, 30 min testing/modification/faculty evaluation)

## Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems. The Supplementary Problems and LeetCode practice set below are for additional practice and are not required for submission.

## Post Laboratory Work (LeetCode Practice)

| Sr. # | LC # | Problem                          | Concept                                          |
|-------|------|----------------------------------|--------------------------------------------------|
| 1     | 933  | Number of Recent Calls           | Queue-based sliding window                       |
| 2     | 346  | Moving Average from Data Stream  | Fixed-size queue processing                      |
| 3     | 239  | Sliding Window Maximum           | Queue-based thinking applied to window traversal |
| 4     | 950  | Reveal Cards in Increasing Order | Queue-based ordering simulation                  |
| 5     | 622  | Design Circular Queue            | Circular array indexing and pointer management   |

## Rubrics (Practical Evaluation / Viva)

| Criteria                                                                                                                                                                    | Weight (%) |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Program correctly handles all queue operations including overflow, underflow, circular indexing in array implementation, and all edge cases in simulation problems.         | 40%        |
| Student can trace through the queue state step by step for a given operation sequence and explain what changes at the front and rear at each step.                          | 25%        |
| Student can explain the difference between array-based and linked-list-based queues and describe the circular indexing problem and how it is resolved.                      | 20%        |
| Code is well-structured with clearly separated functions for each operation and problem, meaningful variable names, and comments explaining front and rear pointer changes. | 15%        |

## Practical 8: Binary Tree Traversals and Binary Search Tree Insertion

|                                                         |                                                                                                                                                                           |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CO Mapping</b>                                       | CO1 – Represent hierarchical (non-linear) data structures in memory;<br>CO2 – Implement non-linear data structure operations and understand their practical applications. |
| <b>Bloom's Taxonomy Level</b>                           | L4 – Analyze                                                                                                                                                              |
| <b>Lab Duration</b>                                     | 4 Hours                                                                                                                                                                   |
| <b>Total Hours of Problem Definition Implementation</b> | 3.5 Hours                                                                                                                                                                 |
| <b>Total Hours of Testing / Evaluation</b>              | 0.5 Hours                                                                                                                                                                 |

### Practical Aim

To implement binary tree traversals and binary search tree insertion, and develop the ability to reason about how the order of visiting nodes changes the information retrieved from a tree.

#### Problem 1

A company stores its organizational hierarchy as a tree — the CEO at the top, managers below, and employees at the leaves. Different departments need the hierarchy printed in different orders: HR wants each manager printed between their left and right sub-teams, the archive department wants each manager printed before their sub-teams, the payroll department wants each manager printed after all their sub-teams are listed, and the floor manager wants everyone printed level by level from top to bottom. Given a binary tree representing the hierarchy, implement all four printing orders and display the result of each.

Describe how your solution for level-by-level printing differs from the other three. What additional structure did you need to use, and why does the same recursive approach that works for the other three not directly apply here?

#### Problem 2

A school library uses a system where every book is assigned a unique code when it arrives. Books are organized so that at any shelf position, all books to the left have smaller codes and all books to the right have larger codes. This arrangement lets staff find any book quickly by always knowing which direction to look. Given a sequence of book codes arriving one by one, insert each into the correct position and print the inorder sequence after all insertions are complete.

Describe how your solution decides where to place each new book code. What property of the final inorder sequence tells you whether all insertions were performed correctly?

### Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] Preorder traversal always visits the root before anything else. Using only this property — without tracing any tree — explain whether it would be possible to reconstruct the original tree from its preorder output alone, and what additional information you would need if it is not.

2. [Problem 2] The inorder traversal of a correctly built BST always produces a sorted sequence. If your inorder output is not sorted after all insertions, what does that tell you about where the error is — in the insertion logic, the traversal logic, or possibly both? How would you isolate which one?

## Supplementary Problems

1. A competition leaderboard stores participant scores in a binary search tree. The organiser needs to find the score that ranks exactly k-th from the bottom — not the lowest, not the highest, but the k-th smallest. Given a BST of scores and a value k, find and print the k-th smallest score.

Describe the approach you used. Which traversal order produces scores in ascending sequence, and how does your solution use that property to identify the k-th smallest? What happens if k is larger than the total number of nodes in the tree?

## Key Skills to be Addressed

Recursive and iterative tree traversal, level-order processing using a queue, BST insertion logic, inorder property of BSTs, and reasoning about how traversal order determines the information retrieved.

## Applications

Organisational hierarchy display, file system navigation, expression tree evaluation, leaderboard and ranking systems, and sorted data retrieval from BSTs.

## Learning Outcome

Students will be able to implement all four binary tree traversals and BST insertion, explain why different traversal orders produce different outputs from the same tree, and use the inorder property of a BST to verify correctness and solve order-based queries.

## Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design suitable test cases covering typical inputs, boundary/edge cases, and at least one stress/worst-case scenario. Document the selected inputs and expected outputs before implementation.

## Tools / Technology to be Used

C / C++, VS Code.

## Total Hours of Engagement

4 Hours (3.5 hrs implementation, 30 min testing/modification/faculty evaluation)

## Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems. The Supplementary Problems and LeetCode practice set below are for additional practice and are not required for submission.

## Post Laboratory Work (LeetCode Practice)

| Sr. # | LC # | Problem                        | Concept                     |
|-------|------|--------------------------------|-----------------------------|
| 1     | 700  | Search in a Binary Search Tree | BST property and navigation |

| Sr. # | LC # | Problem                       | Concept                           |
|-------|------|-------------------------------|-----------------------------------|
| 2     | 104  | Maximum Depth of Binary Tree  | Recursive tree reasoning          |
| 3     | 112  | Path Sum                      | Root-to-leaf reasoning on trees   |
| 4     | 236  | Lowest Common Ancestor of BST | BST structural property reasoning |
| 5     | 98   | Validate Binary Search Tree   | BST property verification         |

### Rubrics (Practical Evaluation / Viva)

| Criteria                                                                                                                                                                                              | Weight (%) |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Program produces correct output for all four traversals and BST insertion, including edge cases such as single node, skewed tree, and duplicate-free insertion.                                       | 40%        |
| Student can trace through any traversal on a given tree and explain why nodes appear in that specific order, and can trace BST insertion step by step identifying which link is created at each step. | 25%        |
| Student can explain why level order traversal requires a different approach from the other three and describe what additional structure is needed and why.                                            | 20%        |
| Code is well-structured with clearly separated functions for each traversal and insertion, meaningful variable names, and comments explaining recursive calls and queue usage.                        | 15%        |

## Practical 9: Graph Traversal: Depth-First Search and Breadth-First Search

|                                                         |                                                                                                                                                                                                         |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CO Mapping</b>                                       | CO1 – Represent graph-based non-linear data structures in memory and analyse traversal strategies;<br>CO2 – Implement non-linear data structure operations and understand their practical applications. |
| <b>Bloom's Taxonomy Level</b>                           | L4 – Analyze                                                                                                                                                                                            |
| <b>Lab Duration</b>                                     | 2 Hours                                                                                                                                                                                                 |
| <b>Total Hours of Problem Definition Implementation</b> | 1.5 Hours                                                                                                                                                                                               |
| <b>Total Hours of Testing / Evaluation</b>              | 0.5 Hours                                                                                                                                                                                               |

### Practical Aim

To implement depth-first and breadth-first traversal on a graph and develop the ability to reason about how the order of exploration changes depending on the traversal strategy used.

#### Problem 1

A disaster relief team needs to inspect all buildings in a city after an earthquake. The city is mapped as a network where buildings are connected by roads. One team starts at a building and goes as deep into each road as possible before backtracking, while a second team spreads out level by level — first checking all buildings directly connected to the start, then all buildings two roads away, and so on. Given the city map as a graph and a starting building, implement both exploration strategies and print the order in which buildings are visited by each team.

Describe how the order of buildings visited differs between the two strategies on the same graph. What data structure does each strategy rely on to decide which building to visit next, and how does that structure cause the difference in order?

### Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] Both strategies visit every reachable node exactly once. If the goal is simply to check whether a specific building exists in the network, which strategy would find it sooner if the building is very close to the start? What if it is very far and deep? Explain your reasoning without tracing through any specific example.
2. [Problem 1] Your DFS implementation likely uses recursion. What real-world constraint on the computer could cause your recursive DFS to fail on a very large city map, and how would you restructure your solution to avoid that failure?

### Supplementary Problems

1. A project manager is planning a sequence of tasks where some tasks depend on others — task A must be done before task B, and so on. Before starting, the manager needs to verify that the dependencies do not form a loop, since a loop would mean a task indirectly depends on itself, making it impossible to start. Given a set of tasks and their

dependencies as an undirected graph, determine whether a loop exists and print Yes or No.

Describe the approach you used to detect the loop. How does your solution avoid mistaking the edge back to a node's direct parent for a loop? What would happen if your solution did not account for that case?

## Key Skills to be Addressed

Graph representation using adjacency list or matrix, depth-first and breadth-first traversal, visited node tracking, traversal order reasoning, and cycle detection in undirected graphs.

## Applications

Navigation and road network systems, social network friend-of-friend exploration, task dependency validation, network packet routing, and disaster relief planning systems.

## Learning Outcome

Students will be able to implement DFS and BFS on a graph, explain how the underlying data structure used by each strategy determines the order of traversal, and apply graph traversal logic to detect structural properties such as cycles in an undirected graph.

## Dataset / Test Data

Use the test data provided in the problem statement, if any. Otherwise, design suitable test cases covering typical inputs, boundary/edge cases, and at least one stress/worst-case scenario. Document the selected inputs and expected outputs before implementation.

## Tools / Technology to be Used

C / C++, VS Code.

## Total Hours of Engagement

2 Hours (1.5 hrs implementation, 30 min testing/modification/faculty evaluation)

## Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems. The Supplementary Problems and LeetCode practice set below are for additional practice and are not required for submission.

## Post Laboratory Work (LeetCode Practice)

| Sr. # | LC # | Problem                      | Concept                             |
|-------|------|------------------------------|-------------------------------------|
| 1     | 733  | Flood Fill                   | DFS on a grid                       |
| 2     | 542  | 01 Matrix                    | BFS level-by-level expansion        |
| 3     | 547  | Number of Provinces          | DFS connected component detection   |
| 4     | 207  | Course Schedule              | Dependency and cycle detection      |
| 5     | 1971 | Find if Path Exists in Graph | Graph reachability using DFS or BFS |

**Rubrics (Practical Evaluation / Viva)**

| Criteria                                                                                                                                                                                                                 | Weight (%) |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Program correctly visits all reachable nodes in the right order for both DFS and BFS, and correctly detects cycles in the supplementary problem, including edge cases such as disconnected nodes and single-node graphs. | 40%        |
| Student can trace through both traversals on a given graph and explain at each step which node is visited next and why, identifying the exact point where DFS and BFS diverge.                                           | 25%        |
| Student can explain in plain English what data structure each traversal relies on internally and describe a real situation where one strategy would find an answer faster than the other.                                | 20%        |
| Code is well-structured with clearly separated functions for each traversal, a visited array used correctly, meaningful variable names, and comments explaining the traversal logic.                                     | 15%        |

## Practical 10: Hash Tables: Linear Probing, Chaining, and Double Hashing

|                                                         |                                                                                                                                                                                                     |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CO Mapping</b>                                       | CO1 – Analyse the upper bound of hash table operations under different collision strategies;<br>CO4 – Implement and analyse hash tables and apply them to real-world dictionary-style applications. |
| <b>Bloom's Taxonomy Level</b>                           | L4 – Analyze / L5 – Evaluate                                                                                                                                                                        |
| <b>Lab Duration</b>                                     | 4 Hours                                                                                                                                                                                             |
| <b>Total Hours of Problem Definition Implementation</b> | 3.5 Hours                                                                                                                                                                                           |
| <b>Total Hours of Testing / Evaluation</b>              | 0.5 Hours                                                                                                                                                                                           |

### Practical Aim

To implement hash tables using different collision handling strategies and develop the ability to reason about why collisions occur, how different strategies resolve them, and what trade-offs each approach involves.

#### Problem 1

A small parking lot has exactly 10 numbered slots (0 to 9). When a vehicle arrives, its registration number is used to assign it a slot by taking the last digit of the number. If that slot is already taken, the vehicle moves to the next available slot in order, wrapping around if necessary. Given a sequence of vehicle registrations, assign each to a slot using this rule and display the final state of all ten slots.

Describe what your solution does when it encounters an already occupied slot. What happens when the parking lot is nearly full and many vehicles keep landing on the same slot? Can you think of a situation where your solution might loop indefinitely?

#### Problem 2

A library uses a system with 10 shelves numbered 0 to 9. Each book's code is used to decide which shelf it belongs to by taking the code modulo 10. Unlike the parking lot, each shelf can hold multiple books stacked together — when two books map to the same shelf, they are simply added to that shelf's stack. Given a sequence of book codes, place each on the correct shelf and display the final contents of all shelves.

Describe how your solution handles two books that map to the same shelf. What happens to search time on a shelf as more and more books pile onto it? Can you think of what would cause all books to end up on the same shelf?

#### Problem 3

A university assigns student IDs to storage slots using a two-step rule. The first rule determines the initial slot. If that slot is taken, instead of simply moving one slot forward, a second rule based on the ID itself determines how many slots to jump. This means different IDs that collide will probe different sequences of slots, spreading them more evenly. Given a sequence of student IDs, assign each to a slot using both rules and display the final state of the table.

Describe how your solution decides how many slots to jump when a collision occurs. What is the risk if the jump size happens to be a factor of the table size? How does this strategy differ from Problem 10.1 in how it spreads collisions?

### **Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation**

1. [Problem 1] If five consecutive vehicle registrations all end in the same digit, trace where each one gets placed in the parking lot. Now explain why this clustering effect is a structural weakness of this collision strategy — not just for this input, but in general.
2. [Problem 2] Suppose all book codes in Problem 2 are multiples of 10. Without tracing through the code, predict what the final shelf layout would look like and explain why. What does this tell you about the relationship between the hash function and the input data?
3. [Problem 3] In Problem 3, what would happen if the second rule always produced a jump size of zero? Trace through the consequence and explain how this failure is related to the choice of the second hash function rather than the data itself.

### **Supplementary Problems**

1. The university administration wants a student records system flexible enough to handle two different scenarios. In departments with few students, records are stored by jumping to the next free slot when a conflict occurs. In departments with many students sharing the same ID range, records at the same slot are chained together. The system must support both modes and allow the administrator to choose which mode to use per department. Given a sequence of student ID and score pairs, implement both modes and display the final table state for each.

Describe how your solution switches between the two modes. When a student record needs to be looked up, does the lookup procedure differ between the two modes? Which mode would perform better if the table is more than 80% full, and why?

### **Key Skills to be Addressed**

Hash function design, collision detection and resolution using linear probing, chaining, and double hashing, trade-off reasoning between collision strategies, and hybrid data structure implementation.

### **Applications**

Database indexing and record lookup, library and inventory management systems, university student record systems, password storage and verification, and cache implementation in operating systems.

### **Learning Outcome**

Students will be able to implement hash tables using three different collision handling strategies, explain why collisions occur and how each strategy resolves them differently, and reason about the conditions under which each strategy performs well or breaks down.

### **Dataset / Test Data**

Use the test data provided in the problem statement, if any. Otherwise, design suitable test cases covering typical inputs, boundary/edge cases, and at least one stress/worst-case scenario. Document the selected inputs and expected outputs before implementation.

## Tools / Technology to be Used

C / C++, VS Code.

## Total Hours of Engagement

4 Hours (3.5 hrs implementation, 30 min testing/modification/faculty evaluation)

## Post Laboratory Work Description

Submit source code, sample outputs/screenshots, and written answers to the Key Questions above for the main problems. The Supplementary Problems and LeetCode practice set below are for additional practice and are not required for submission.

## Post Laboratory Work (LeetCode Practice)

| Sr. # | LC # | Problem                      | Concept                                  |
|-------|------|------------------------------|------------------------------------------|
| 1     | 706  | Design HashMap               | Hash table implementation from scratch   |
| 2     | 380  | Insert Delete GetRandom O(1) | Hash table with O(1) operations          |
| 3     | 49   | Group Anagrams               | Hash function applied to string grouping |
| 4     | 128  | Longest Consecutive Sequence | Hash set for O(1) lookup                 |
| 5     | 454  | 4Sum II                      | Multi-map hashing across datasets        |

## Rubrics (Practical Evaluation / Viva)

| Criteria                                                                                                                                                                                      | Weight (%) |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Program correctly inserts elements using all three collision strategies, handles edge cases such as full table, all-same-slot inputs, and zero jump size in double hashing.                   | 40%        |
| Student can explain step by step how each collision strategy resolves a conflict differently, and identify which specific decision in the code determines the next slot to probe.             | 25%        |
| Student can describe a concrete input scenario where one collision strategy would clearly outperform another, and articulate the structural reason behind that difference.                    | 20%        |
| Code is well-structured with clearly separated functions for each strategy, meaningful variable names, and comments explaining the hash function and collision resolution logic at each step. | 15%        |